

LEAPER PROJECT

Blog Backbone

A story structure for writing about an artist learning game AI, reinforcement learning, experimentation, and technical language through the Leaper project.

How an artist with almost no technical background learned to design, build, test, misunderstand, diagnose, and gradually improve a reinforcement-learning experiment.

Central perspective

- You began as an artist interested in how a game creature could appear intelligent.
- You did not know the technical vocabulary, so every unfamiliar word became a separate lesson.
- Codex helped build and inspect the project. Claude became a translator used to understand Codex's explanations.
- You contributed through visual judgment, game-design questions, observation, and repeatedly challenging explanations that did not match what you saw.
- The project did not move in a straight line. Wrong assumptions, failed experiments, and corrections are the substance of the story.

Suggested story structure

1. The creature that started the question

- Discovering the Leaper-like enemy concept.
- Wondering whether a game enemy thinks.
- Starting with ideas such as pursuit, hiding, shooting, injury, recovery, water hazards, vision loss, and destructible body parts.
- The original question: how would I create a creature with believable decisions?

Main lesson: Believable behavior is designed, even when it is learned.

2. My first misunderstanding: game AI is not automatically RL

- Learning what behavior trees and state machines are.
- Understanding that most game enemies follow authored rules.
- Behavior tree: the designer writes the decisions. Reinforcement learning: the designer defines the situation, available actions, and consequences.

- Choosing RL because learning and emergent behavior were part of the project's purpose, not because RL was the easiest way to make an enemy.

Main lesson: RL was an experimental and educational choice, not the default solution.

3. Translating RL into an artist's language

- Agent: the learner. Environment: its world. Observations: its senses.
- Actions: what its body is allowed to do. Reward: the feedback shaping its behavior.
- Policy: the strategy it learns. PPO: the ready-made learning method under the project.
- Step, episode, training rollout, and evaluation become practical rather than abstract words.
- Observations, actions, and rewards become character-design tools: sculpting behavior with numbers.

Main lesson: The technical vocabulary becomes easier when every term is tied to something visible.

4. Defining the first achievable Leaper

- Cutting away the imagined full combat creature.
- No shooting, damage system, hiding, jumping, or moving player yet.
- One stationary pink target, a bounded arena, obstacles, and a Leaper allowed to move and turn.
- One job: reach the target.

Main lesson: Save the full character fantasy, but prove one behavior first.

5. Why we built a proof of concept on the web

- Comparing a technical proof of concept to blocking out a 3D scene.
- Beginning with a fast browser prototype instead of a finished game.
- Building a procedural 3D hexapod and arena in Three.js.
- Starting with one large HTML prototype, then separating the robot, world, cameras, controls, simulation, and interface into modules.
- Using Git as a project-wide history and safety net.

Main lesson: The rough version was the blockout that made later decisions possible.

6. The important truth beneath the 3D appearance

- Visually, Leaper is a six-legged 3D robot with a reconstructed walking gait.
- Underneath, Python currently learns simplified flat navigation: signed throttle and turning.
- There is no learned joint control, balance, jumping, or falling.
- The legs later became simplified collision sample points rather than learned limbs.

Main lesson: The 3D body is real, but the learned problem is currently navigation. Do not oversell it as learned hexapod locomotion.

7. Creating two connected worlds

- JavaScript owns the visible simulator, controls, terrain, animation, and training replay.
- Python owns the RL environment, PPO training, evaluation, and diagnostics.
- The visible JavaScript world cannot simply be exported into Python; its rules are rebuilt in simplified form.
- Movement, actions, observations, timing, obstacles, collisions, and success need an explicit shared contract.

Main lesson: The trained brain only makes sense if the world where it learned agrees with the world where it will be used.

8. Teaching with rewards, and discovering that rewards can lie

- Beginning with a reward only at the destination, which provided almost no guidance.
- Adding progress reward: $0.2 \times (\text{previous distance} - \text{current distance})$.
- Adding -0.01 per step, -0.18 for collision, and +25 for reaching the target.
- Questioning whether moving temporarily away from the target unfairly punishes a correct detour.
- Exploring straight-line versus geodesic distance and flood-fill, then deciding geodesic distance was not yet necessary for the sparse arena.

Main lesson: Reward shaping should guide gently. It should not secretly prescribe one path or make exploration too painful.

9. Making training visible instead of trusting a number

- Printing episode results and adding TensorBoard graphs.
- Building a dedicated browser training view with checkpoint and real-rollout replay.
- Adding scrubbing, worker and episode selection, imports, and a live JSON bridge.
- Separating exploratory training behavior from deterministic evaluation.

Main lesson: Watching one promising replay is not the same as proving the policy works reliably.

10. The experiment history: success, regression, and diagnosis

- PPO_1 was the initial technical smoke test.
- PPO_2 became the strongest known body-only run: 60% deterministic success at 300,000 steps.
- PPO_3 and PPO_4 showed that adding leg collision points made the task dramatically harder.
- PPO_5 through PPO_7 explored diagnostic and reward directions, including incomplete or unsuccessful experiments.
- PPO_8 used cleaner distance-change shaping but finished with 0% deterministic success after 300,000 requested steps.

Main lesson: A simpler version succeeding does not guarantee that a more demanding version will inherit that success.

11. The zero-throttle mystery

- Zero throttle meant Leaper's forward command was almost always zero.
- Moving exposed it to collisions and penalties; standing still produced a smaller predictable loss.
- Diagnostics proved the target was reachable within 400 steps and the +25 target reward was already large enough.
- More training and a larger target reward were not the answer.

Main lesson: When an agent behaves badly, first ask how that behavior makes sense under the rules it was given.

12. Learning to change one thing at a time

- PPO_9 added the previous collision, forward action, and turn action to the observation.
- Exploratory behavior improved, but the deterministic policy still froze.
- PPO_10 changed only the forward action from [0, 1] to signed [-1, 1], allowing reverse movement.
- The action-language change made older trained policies incompatible.

Main lesson: A failed controlled experiment is useful because it removes one possible explanation.

13. PPO_10: fixing the freeze but exposing the next problem

- Deterministic stopped steps fell from effectively 100% to 0%.
- Leaper used both forward and reverse and made meaningful target progress.
- Deterministic success rose to 5%; stochastic training success reached 34%, but these are different measurements.
- Collision behavior became severe, including very long collision streaks.

Main lesson: Solving one failure can uncover the next one. Movement returned, but reliable obstacle recovery did not.

14. Where the project honestly stands

- A maintainable browser simulator, Python PPO pipeline, replays, checkpoints, diagnostics, and experiment ledger now exist.
- The system can distinguish training excitement from deterministic reliability.
- Dependable navigation with the current collision system is not solved.
- The next responsible test is repeating PPO_10 with two new seeds.
- If the result does not repeat, the next controlled idea is separating rotation and translation collision resolution.

Main lesson: The project is a functioning experimental platform with a clearly diagnosed next question, not a finished intelligent creature.

Closing themes for the eventual blog

- Technical language became less frightening once every word was tied to something visible.
- An artist's observational skills were directly useful in debugging behavior.
- Asking why a behavior made sense to Leaper was more useful than treating it like a disobedient character.
- Failed runs produced knowledge when they were documented and compared carefully.
- The greatest progress was not one success percentage; it was learning how to stop guessing.
- The project changed from make the spider reach the pink box into learning how to conduct a controlled experiment.

Facts the finished blog must not misstate

- Leaper is not currently learning individual leg control or a physical gait.
- The working system is Three.js plus Python, Gymnasium, Stable-Baselines3, and PPO; it is not Unity ML-Agents.
- PPO_2's authoritative deterministic result is 60% at the 300k checkpoint, not simply 80% because a graph appeared near 0.8.
- PPO_10's 34% stochastic training success must remain separate from its 5% deterministic success.
- A larger target reward was considered and rejected after diagnostics showed +25 already dominated individual penalties.
- Geodesic distance was an important learning discussion, not a feature that was ultimately implemented.
- Signed throttle fixed literal freezing, but it did not solve collision recovery.

Project references

The factual experiment record is maintained in TRAINING.md, with practical setup in README-RL.md. The learning trail came from three shared conversations covering the project's origin, reward and training discussions, and documentation/report interpretation.

Prepared from the Leaper repository and its experiment ledger. August 2026.